KLUDGE WAGON HAYWIRE · SR-404 SERVICE CARD

C Integers, Floats & Undefined Behaviour

The throttle, steering and GPS maths all ride on C's arithmetic rules: promotions, truncation, wraparound and the optimiser's assumptions about UB.

Debug build:
`-std=c17 -Wall -Wextra -g -fsanitize=address,undefined`
Also try: `-O2` · `-fanalyzer` · `-fsanitize=thread`

bug example (see captured check) · fixed example (checks have bounded scope) · ▸ = what gcc (GCC) 16.2.1 20260810 / its sanitizers actually reported · 13 entries

C-INT-01 Signed integer overflow

SR-404 · Odometer

✕ BUGGY
```c
int odo_add(int odo, int miles) {
    return odo + miles;  // core C overflow rule; compiler flags matter
}
```

▸ UBSan: signed integer overflow: 2147483647 + 1 cannot be represented…

What goes wrong: C11 core signed overflow is undefined. Optimizers can use the no-overflow premise; a particular folded comparison or deleted check depends on the compiler and options.

^ FIX
```c
bool odo_add(int odo, int miles, int *out) {
    if ((miles > 0 && odo > INT_MAX - miles) ||
        (miles < 0 && odo < INT_MIN - miles))
        return false;          // would overflow: report it
    *out = odo + miles;
    return true;
}
```

Sources: C11 draft 6.5p5; GCC14 docs Checked arithmetic builtins; Clang docs Checked Arithmetic Builtins; C23 draft N3096 7.20.1; GCC14 docs ftrapv; fwrapv; Host example capture C-INT-01 exact supplied-example diagnostics/results

Avoid it: Check representability before adding, or use a supported checked-arithmetic facility. GCC/Clang provide overflow builtins; C23 provides `ckd_add` from `<stdckdint.h>`. GCC14 `-ftrapv` / `-fwrapv` alter addition, subtraction and multiplication handling, not every operation.

C-INT-02 Integer promotion: narrow unsigned may become signed

SR-404 · Wheel-speed sensor

✕ BUGGY
```c
uint16_t raw = 65535;
uint32_t sq = raw * raw;  // on the recorded target, both promote to int:
                          // product exceeds INT_MAX → UB
printf("%" PRIu32 "\n", sq);
```

▸ UBSan: signed integer overflow: 65535 * 65535 cannot be represented …

What goes wrong: Integer promotions happen before arithmetic. With the recorded 32-bit int, uint16_t values promote to int and 65535 squared is unrepresentable. The complement result -16 additionally assumes the shown two's-complement representation.

^ FIX
```c
uint16_t raw = 65535;
uint32_t sq = (uint32_t)raw * raw;  // unsigned 32-bit on the recorded target
printf("%" PRIu32 "\n", sq);
```

Avoid it: Choose an operation type with sufficient range after promotions, and convert before multiplying. Shifts also require a valid count. Requested conversion warnings and sanitizer checks are not complete proofs.

Sources: C11 draft 6.3.1.1;6.3.1.8;6.2.6.2;6.5.3.3; GCC14 docs signed-integer-overflow; GCC14 docs Wconversion; Wsign-conversion; Host example capture C-INT-02 exact supplied-example diagnostics/results; Defined host checks host_abi: actual widths/radix/double comparison; not full AnnexF conformance

C-INT-03 Unsigned wraparound in a countdown loop

SR-404 · Radio text scroller

✕ BUGGY
```c
void print_rev(const char *s) {
    for (size_t i = strlen(s) - 1; i >= 0; i--)  // always true
        putchar(s[i]);  // "" → i = SIZE_MAX
}
```

▸ gcc -Wtype-limits: comparison of unsigned expression in '>= 0' is al…

What goes wrong: An unsigned value cannot be negative, so i >= 0 cannot end this loop. Converting -1 to size_t yields SIZE_MAX. The type of strlen(s) - 1 itself depends on integer promotions.

^ FIX
```c
void print_rev(const char *s) {
    for (size_t i = strlen(s); i-- > 0; )  // test, then decrement
        putchar(s[i]);
}
```

Avoid it: The shown i-- > 0 loop avoids an empty-string access, but i can wrap after the final test. Rewriting a - b < c as a < b + c requires a >= b and representable b + c for equivalence.

Sources: C11 draft 6.2.5p9;6.3.1.1;6.3.1.8;6.5.2.4;7.19; GCC14 docs Wtype-limits; Wextra; Host example capture C-INT-03 exact supplied-example diagnostics/results

C-INT-04 Shift too far, or into the sign bit

SR-404 · Lamp bitmask

✕ BUGGY
```c
uint32_t bit_mask(int bit) {
    return 1 << bit;  // 1 is int; count/value constraints depend on promoted int
}
```

▸ UBSan: shift exponent 32 is too large for 32-bit type 'int'

What goes wrong: A negative count or a count at least the promoted left operand's width is undefined in C11. Signed left shift also has value constraints. Negative signed right shift is implementation-defined; no particular C result follows from hardware count masking.

^ FIX
```c
uint32_t bit_mask(unsigned bit) {
    return bit < 32 ? UINT32_C(1) << bit : 0;  // range-checked
}
```

Avoid it: Check the promoted operation type, value range and count. UINT32_C corresponds to promoted uint_least32_t, not necessarily an exact 32-bit unsigned operation. GCC14 and Clang document shift sanitizers; GCC extensions are separate from ISO rules.

Sources: C11 draft 6.5.7;7.20.4;7.20.4.1; GCC14 docs Signed shift extension and diagnosis; Clang docs shift checks; Host example capture C-INT-04 exact supplied-example diagnostics/results

C-INT-05 Division by zero (and INT_MIN / -1)

SR-404 · Average RPM

✕ BUGGY
```c
int avg_rpm(int total, int count) {
    return total / count;  // zero divisor: core C undefined behavior
}
```

▸ UBSan: division by zero

What goes wrong: Integer division needs a nonzero divisor and a representable quotient. INT_MIN / -1 is invalid on an asymmetric signed range. A required process signal is not an ISO C guarantee. Floating zero-divisor behavior depends on the language/IEC60559 contract and environment.

^ FIX
```c
bool avg_rpm(int total, int count, int *out) {
    if (count == 0 || (total == INT_MIN && count == -1))
        return false;  // reject zero; conservatively reject INT_MIN / -1
    *out = total / count;
    return true;
}
```

Avoid it: Check external divisors and quotient range before dividing. Use documented executed-case checks, not a promised outcome from undefined behavior. isfinite detects nonfinite results when the floating environment permits returning them.

Sources: C11 draft 6.5.5p5-p6;Annex F.1/F.3/F.7;7.12.3.2; GCC14 docs integer-divide-by-zero; Host example capture C-INT-05 exact supplied-example diagnostics/results

C-INT-06 Integer division truncates

SR-404 · Throttle (Ep 3)

✕ BUGGY
```c
float throttle_cmd(int pedal_mm, int travel_mm) {
    return pedal_mm / travel_mm;   // 30 / 40 → 0, not 0.75
}
```

▸ prints: throttle=0.00

What goes wrong: Defined integer division discards the fraction before a later float conversion. The shown 30 / 40 becomes integer 0 before conversion, whereas floating division gives 0.75; this is an example, not a measured whole-car response.

^ FIX
```c
float throttle_cmd(int pedal_mm, int travel_mm) {
    if (travel_mm <= 0) return 0.0f;
    return (float)pedal_mm / (float)travel_mm;  // 30 / 40 → 0.75
}
```

Avoid it: Convert an operand before dividing and inspect integer intermediates. The named clang-tidy bugprone-integer-division check has documented exclusions. Test fractional inputs; a tool warning is not proof of all-input correctness.

Sources: C11 draft 6.5.5p6;6.3.1.4;6.3.1.8; Clang docs check description and exclusions; Host example capture C-INT-06 exact supplied-example diagnostics/results

C-INT-07 Strict aliasing (type punning via pointer cast)

SR-404 · CAN frame decoder

✕ BUGGY
```c
float f = 1.0f;
uint32_t bits = *(uint32_t *)&f;  // invalid access on the shown target
printf("%08" PRIx32 "\n", bits);
```

▸ gcc -Wstrict-aliasing: dereferencing type-punned pointer will break …

What goes wrong: C11 effective-type rules have specified exceptions. On the captured target, accessing this float through uint32_t * violates them. Optimization can expose the invalid access; alignment and object validity must also hold.

> FIX
```c
float f = 1.0f;
_Static_assert(sizeof(float) == sizeof(uint32_t), "32-bit float required");
uint32_t bits;
memcpy(&bits, &f, sizeof bits);  // copies representation; no instruction-count promise
printf("%08" PRIx32 "\n", bits);
```

Sources: C11 draft 6.5p7;6.3.2.3p7;6.2.6.1;7.24.2.1; C++20 draft N4861 [bit.cast] constraints; GCC14 docs Wstrict-aliasing; GCC14 docs fstrict-aliasing; Host example capture C-INT-07 exact supplied-example diagnostics/results

Avoid it: Use memcpy with size/representation checks; exact hex also needs byte-order assumptions. C++20 bit_cast requires equal size and trivially copyable types. GCC strict-aliasing diagnostics require active `-fstrict-aliasing`; a flag change does not repair unrelated errors.

C-INT-08 Unsequenced modifications (i++ + i++)

SR-404 · Sample buffer

✕ BUGGY
```c
int i = 1;
int x = i++ + i++;        // two unsequenced writes to i: UB
int buf[4] = {0};
buf[i] = i++;  // UB in C; C++17 uses the incremented index
printf("%d %d\n", x, buf[2]);
```

▸ gcc -Wsequence-point: operation on 'i' may be undefined

What goes wrong: i++ + i++ is unsequenced in C11 and C++17. Assignment sequencing differs: an isolated C++17 buf[i] = i++ uses the incremented index and can access buf[4] when i starts at 3. Earlier undefined behavior does not define a later whole-program trace.

> FIX
```c
int i = 1;
int x = i + (i + 1);      // what was meant, with no side effects
i += 2;
int buf[4] = {0};
buf[i] = i;               // index and value from the same i
i++;
printf("%d %d\n", x, buf[3]);
```

Avoid it: Separating updates and accesses is conservative style guidance, not a language mandate for every expression. GCC `-Wsequence-point` and Clang `-Wunsequenced` request diagnostics. Review macros that duplicate argument evaluation.

Sources: C11 draft 6.5p2;6.10.3.1; C++17 draft N4659 [intro.execution];[expr.ass]; GCC14 docs Wsequence-point; Clang docs Wunsequenced; Host example capture C-INT-08 exact supplied-example diagnostics/results

C-INT-09 Signed/unsigned comparison and silent narrowing

SR-404 · Steering damping (Ep 2)

✕ BUGGY
```c
int temp_c = -5;
unsigned limit = 100;
if (temp_c < limit)  // same-rank int/unsigned int conversion
    puts("in range");            // never printed
int16_t rpm = read_rpm();  // out-of-range signed conversion: implementation-defined
```

▸ gcc -Wsign-compare: comparison of integer expressions of different s…

What goes wrong: Mixed integer operands follow rank and representability rules; they do not always convert to unsigned. Out-of-range signed integer conversion is implementation-defined or can signal in C11. Floating truncation is a separate rule, not dropping high integer bits.

> FIX
```c
int temp_c = -5;
unsigned limit = 100;
if (temp_c >= 0 && (unsigned)temp_c < limit)
    puts("in range");
int raw = read_rpm();
int16_t rpm = (int16_t)(raw > INT16_MAX ? INT16_MAX
                        : raw < 0 ? 0 : raw);   // saturate
```

Avoid it: Validate the source value and destination range before casting. The shown RPM clamp is this exercise's chosen nonnegative range. GCC14 `-Wconversion`/`-Wsign-conversion` request selected implicit-conversion diagnostics; explicit casts are not validation.

Sources: C11 draft 6.3.1.3;6.3.1.4;6.3.1.8; GCC14 docs Wsign-compare;Wconversion;Wsign-conversion; Host example capture C-INT-09 exact supplied-example diagnostics/results

C-INT-10 Comparing floats with ==

SR-404 · Fuel gauge

✕ BUGGY
```c
double v = 0.1 + 0.2;
if (v == 0.3)  // captured binary64 target: comparison is false
    puts("equal");
```

▸ gcc -Wfloat-equal: comparing floating-point with '==' or '!=' is uns…

What goes wrong: These fractions are not exact finite binary fractions; the captured binary example compares unequal after rounding. ISO C need not use a binary radix. Under the IEC60559 quiet-NaN contract, a NaN never compares equal, even to itself.

> FIX
```c
double v = 0.1 + 0.2;
if (fabs(v - 0.3) <= 1e-9 * fmax(1.0, fabs(v)))  // tolerance
    puts("equal");
```

Avoid it: Choose equality or an error tolerance for the actual numerical contract. isnan detects NaNs; isinf detects infinities; isfinite rejects either nonfinite class. GCC14 `-Wfloat-equal` requests floating-equality diagnostics, not every == or a correctness proof.

Sources: C11 draft 5.2.4.2.2;Annex F.2.1/F.3;7.12.3.2-7.12.3.4; Goldberg/Oracle Binary representation and error; GCC14 docs Wfloat-equal; Host example capture C-INT-10 exact supplied-example diagnostics/results; Defined host checks host_abi; actual widths/radix/double comparison; not full AnnexF conformance

C-INT-11 Out-of-range float → int conversion

SR-404 · Steering rack

✕ BUGGY
```c
int steer_counts(double angle) {
    return (int)(angle * 1000.0);  // out-of-range conversion; see core/Annex F rules
}
```

▸ UBSan: 1e+10 is outside the range of representable values of type 'i…

What goes wrong: The C11 core out-of-range floating conversion rule differs from optional Annex F: Annex F raises invalid and gives an unspecified result. Neither contract guarantees a portable integer result or rack motion.

> FIX
```c
int steer_counts(double angle) {
    if (!isfinite(angle)) return 0;
    if (angle > 30.0) return 30000;
    if (angle < -30.0) return -30000;
    return (int)(angle * 1000.0); // bounded before scaling
}
```

Avoid it: Check finiteness and bound the input before scaling or converting. The fixed counts range is an exercise policy. GCC14 excludes float-cast-overflow from undefined; retrieved Clang documentation includes it. Name the compiler and requested check.

Sources: C11 draft 6.5p5;6.3.1.4;Annex F.1/F.4; GCC14 docs float-cast-overflow groups; Clang docs float-cast-overflow groups; Host example capture C-INT-11 exact supplied-example diagnostics/results

C-INT-12 Degrees passed to sin/cos

SR-404 · GPS / minimap side quest

✕ BUGGY
```c
double lon_scale(double lat_deg) {
    return cos(lat_deg);  // cos() takes radians: 37 rad, not 37°
}
```

▸ prints: 0.765

What goes wrong: sin, cos and tan take angle arguments in radians. atan2 takes y then x and returns the principal angle in radians for valid numeric inputs. A particular minimap distortion requires a caller/projection model.

> FIX
```c
static const double kPi = 3.141592653589793;  // M_PI isn't ISO C
double lon_scale(double lat_deg) {
    return cos(lat_deg * kPi / 180.0);   // 37° → 0.799
}
```

Avoid it: Make units explicit and convert at boundaries. Test known mathematical values with an appropriate floating error policy: cos 60 degrees is mathematically 0.5. M_PI is specified by POSIX/XSI, not required by ISO C11.

Sources: C11 draft 7.12.4.4-7.12.4.7;7.12; POSIX math M_PI, XSI option; Host example capture C-INT-12 exact supplied-example diagnostics/results

C-INT-13 % keeps the sign of the dividend

SR-404 · GPS turn arrow

✕ BUGGY

```
int wrap_heading(int deg) {
    return deg % 360;          // -350 % 360 == -350, not 10
}
```

▸ prints: -350

What goes wrong: For defined integer division, a nonzero remainder has the dividend's sign. fmod has related sign behavior for valid finite inputs, but different domains and exceptions. A negative remainder alone does not establish a nav-arrow motion.

> FIX

```
int wrap_heading(int deg) {
    int r = deg % 360;
    return r < 0 ? r + 360 : r;   // always 0..359
}
```

Avoid it: For positive m and a valid division, r < 0 ? r + m : r gives a nonnegative representative. Test negative and half-turn inputs. remainder(x,360.0) permits both +180 and -180 by its ties-to-even quotient rule; choose the intended endpoint policy.

Sources: C11 draft 6.5.5p6;7.12.10.1-7.12.10.2;footnote239; Host example capture C-INT-13 exact supplied-example diagnostics/results

Bug examples were compiled; runnable cases were executed on 2026-10-27T00:32:15+0000 with gcc (GCC) 16.2.1 20260810 (C17). Full sources, including the hidden test drivers: `cheatsheets/src/snippets/<ID>/` . Mistake IDs match `docs/mistakes_catalog.json`, so in-game bugs can link here. Rendering does not repeat compilation. Public drafts are identified in the references; passing checks do not prove all-input correctness. Author correction draft: independent content review remains pending.